Account

1. Feature Overview

Open accounts under a customer, enforce account policies (opening deposit / min balance metadata), search accounts, update status, and post simple deposits that credit balances.

Status: Implemented for open / search / get-by-id / status / deposit / withdraw / **transfer** / policies / transaction list API. Deposit and **opening deposit** (`openAccount` when amount > 0) write a **CREDIT** row; **withdraw** writes a **DEBIT** row; **transfer** writes DEBIT+CREDIT via `TransactionService.record`. Withdraw uses `findByIdForUpdate` (pessimistic lock) and rejects insufficient funds. **Transfer** locks both accounts in id order, rejects same-account / currency mismatch / insufficient funds; writes DEBIT + CREDIT ledger rows. **Notifications:** after open/deposit/withdraw/transfer, `NotificationEventPublisher` sends Kafka events; email goes to the **customer email** (see Notification.md).

2. Business Purpose

Hold customer balances and product types (SAVINGS, CURRENT, SALARY, FD, RD, LOAN) with branch-coded account numbers.

3. User Workflow

1. `/app/accounts` — search list, Deposit / Withdraw / Transfer actions
2. Customer detail — Add Current (loads policy → opening deposit dialog if required) / Add Loan
3. Customer create — optional first account
4. Policies seeded at startup; create policy via API

4. Execution Flow

See CALL_FLOW.md, deposit, get active account policy.

5. Database Tables

Table	Purpose
`account`	Balances, status, number parts
`account_policy`	Per type+currency rules
`account_ordinal_seq`	Ordinal for number generation
`customers`	Owner FK

6. REST APIs

Method	Path	Roles
POST	`/accounts`	ADMIN, EMPLOYEE
GET	`/accounts`	ADMIN, EMPLOYEE, MANAGER
GET	`/accounts/customer/{customerId}`	ADMIN, EMPLOYEE, MANAGER
GET	`/accounts/{accountId}`	ADMIN, EMPLOYEE, MANAGER
GET	`/accounts/{accountId}/transactions`	ADMIN, EMPLOYEE, MANAGER
PUT	`/accounts/{accountId}/status`	ADMIN, EMPLOYEE
POST	`/accounts/{accountId}/deposit`	ADMIN, EMPLOYEE
POST	`/account-policies`	Authenticated*
GET	`/account-policies?accountType&currencyCode`	Authenticated*

*Restrict to ADMIN — deferred hardening.

7. Controllers

- `AccountController`
- `AccountPolicyController`

8. Services

- `AccountService` / `AccountServiceImpl` — `openAccount`, `getAccountById`, `getAccountsByCustomerId`, `updateAccountStatus`, `deposit`, `searchAccounts`; transactions via `TransactionService.getByAccountId`
- `AccountPolicyService` / `AccountPolicyServiceImpl` — `createPolicy`, `getActivePolicy`
- `AccountPolicyInitializer` — seed INR policies

9. Repositories

- `AccountRepository` (+ `getNextOrdinal()` native query)
- `AccountPolicyRepository`

10. DTOs

`OpenAccountRequest`, `AccountResponse`, `UpdateAccountStatusRequest`, `DepositRequest`, `CreateAccountPolicyRequest`, `AccountPolicyResponse`

11. Entities / Enums

- `Account`, `AccountPolicy`
- `AccountType`, `AccountStatus`, `CurrencyCode`

12. Utility Classes

- `AccountNumberGenerator.generate(...)`
- `AccountSpecification.matching(...)`

13. Configuration Classes

`SecurityConfig` matchers for `/accounts/**`; policy paths fall under `anyRequest().authenticated()`.

Frontend: `core/services/account.ts`, `account-policy.ts`, `features/accounts/`, `features/opening-deposit-dialog/`

14. Validation Rules

- Open: customer exists; active policy in effective window; if `openingDepositRequired` then amount ≥ `requiredOpeningDeposit`
- Deposit: account ACTIVE; amount positive
- Policy create: Bean Validation on `CreateAccountPolicyRequest`

15. Security Rules

Account mutations ADMIN/EMPLOYEE; reads include MANAGER. Policy endpoints currently any authenticated user.

16. Exception Handling

Business exceptions for missing account/customer/policy and validation failures → `GlobalExceptionHandler`.

17. Logging

SQL debug in dev; no structured account event log.

18. Audit Events

Account has `created_by` / timestamps; `AccountPolicy` uses `AuditableEntity`. Opening deposits (> 0) and post-open deposits write an immutable `bank_transaction` CREDIT via `TransactionService.record`.

19. Testing Strategy

- Open CURRENT with deposit < 5000 → reject
- Open CURRENT with ≥ policy min → balances equal deposit
- Deposit on ACTIVE → balances and `creditCount` increase
- Deposit on non-ACTIVE → reject
- GET policy CURRENT/INR returns seed row

- UI: Add Current opens dialog when required

20. Future Extension Guide

- Status transition state machine (no CLOSED→ACTIVE; close with zero balance)
- Seed LOAN policy or block LOAN until product module exists
- Maker–checker for large deposits
- Notify **both** parties on transfer; Kafka outbox
- Daily interest / statement PDF

Future Modification Guide

Requirement: Increase required opening deposit for CURRENT/INR

Item	Detail
Prefer	Update/insert `account_policy` row (API `POST /account-policies` or DB) — **not** hardcode UI
Files	`AccountPolicyInitializer.java` (seed), or runtime policy via `AccountPolicyServiceImpl.createPolicy`
Classes	`AccountPolicy`, `AccountPolicyServiceImpl`
Methods	`createPolicy()`, `getActivePolicy()`; enforced in `AccountServiceImpl.openAccount()`
Impact	`OpeningDepositDialog` min; customer onboarding amounts

Requirement: Change deposit to write a ledger entry

Done: deposit, openAccount (opening CREDIT), withdraw (DEBIT), transfer (DEBIT+CREDIT), account-detail ledger UI. Still TODO: dashboard `todayTransactionCount`.

Item	Detail
Files	`AccountServiceImpl.java`, `com/bankone/transaction/**`
Methods	`deposit()` → `transactionService.record(...)`
Impact	Ledger table populated on deposit; dashboard count still stub

Requirement: Change account number format

Item	Detail
Files	`AccountNumberGenerator.java`, possibly `AccountServiceImpl.openAccount`
Methods	`generate(branchCode, accountType, ordinal, currency, checkDigit)`
Impact	Display/search only for new accounts; old numbers unchanged

Requirement: Allow deposit on FROZEN accounts

Item	Detail
Files	`AccountServiceImpl.java`
Methods	`deposit()` status check
Impact	Status rules doc; UI Deposit button enablement

Call hierarchy (open account)

```
AccountController.openAccount()
        ↓
AccountServiceImpl.openAccount()
        ↓
CustomerRepository.findById()
        ↓
AccountPolicyRepository.findByAccountTypeAndCurrencyCodeAndActiveTrue()
        ↓
AccountRepository.getNextOrdinal()
        ↓
AccountNumberGenerator.generate()
        ↓
AccountRepository.save()
```

Call hierarchy (deposit)

```
AccountController.deposit()
        ↓
AccountServiceImpl.deposit()
        ↓
AccountRepository.findById()
        ↓
TransactionService.record(..., CREDIT, ...)
        ↓
NotificationEventPublisher.publish(DEPOSIT_COMPLETE, recipientEmail)
        ↓
Kafka → NotificationEventConsumer → email (SMTP/SendGrid)
```